

Datamak Multi-User Configuration

Updated: 2026-03-26

Contents

1 Purpose	1
2 Design Overview	2
3 User-Local Config File	2
3.1 Location	2
3.2 Schema	2
4 Resolved Defaults	3
4.1 Perlmutter	3
4.2 Flux	3
4.3 Source metadata	3
5 Environment Variables	3
6 GUI Workflow	4
7 CLI and Shell Workflow	4
7.1 Python entrypoints	4
7.2 Shell helper	4
8 Flux Runtime Hand-Off	4
9 Perlmutter Runtime Behavior	5
10 Migration from Legacy Repo-Local Config	5
11 Typical Setup for a New User	5
12 Troubleshooting	6
13 Scope Note	6

1 Purpose

This document describes how Datamak was upgraded from a single-user repository layout to a multi-user workflow. It covers both implementation details and the operator workflow for configuring the application on a new account.

2 Design Overview

The main design goal is to remove assumptions that the active user is a specific person or that runtime data always lives in a single personal directory tree.

The upgrade introduces one shared configuration resolver in `dtwin_config.py` and one shell-oriented helper in `tools/resolve_dtwin_env.py`. Active scripts now resolve settings in the same order:

1. explicit CLI arguments,
2. environment variables,
3. user-local Datamak config,
4. derived defaults for standard Perlmutter and Flux layouts,
5. hard failure when a required value still cannot be inferred.

3 User-Local Config File

3.1 Location

Datamak reads the user config from:

1. `DTWIN_CONFIG`, if set,
2. `$XDG_CONFIG_HOME/datamak/config.json`, if `XDG_CONFIG_HOME` is set,
3. `$HOME/.config/datamak/config.json` otherwise.

This keeps per-user workflow settings outside the repository checkout.

3.2 Schema

```
{
  "version": 1,
  "perlmutter": {
    "user": "",
    "host": "perlmutter.nersc.gov",
    "identity": "",
    "control_path": "/tmp/datamak_ssh_%r@%h_%p",
    "control_persist": "10m",
    "connect_timeout": 10,
    "monitor_timeout": 120,
    "base_dir": "",
    "batch_dir": "",
    "gx_path": ""
  },
  "flux": {
    "user": "",
    "host": "flux",
```

```

    "base_dir": "",
    "python_bin": ""
}
}

```

4 Resolved Defaults

4.1 Perlmutter

When Perlmutter settings are not explicitly configured, Datamak derives:

- user from `$USER`,
- host as `perlmutter.nersc.gov`,
- base directory as `/pscratch/sd/<first-letter>/<user>/DTwin`,
- batch directory as `<base-dir>/newbatch`,
- GX path as `/global/homes/<first-letter>/<user>/GX/gx_next6`.

4.2 Flux

When Flux settings are not explicitly configured, Datamak derives:

- user from `$USER`,
- host as `flux`,
- base directory as `/u/<user>/DTwin/transp_full_auto`,
- Flux Python as `/u/<user>/pyrokinetics/.venv/bin/python`.

4.3 Source metadata

Source definitions such as Mate roots or TRANSP copy locations are not part of the GUI-managed multi-user settings contract. They belong to the workflow data model and should be represented through database records such as `data_origin`. The per-user settings window therefore does not edit them.

5 Environment Variables

The main environment variables are:

- `DTWIN_CONFIG`
- `DTWIN_PERLMUTTER_USER`, `DTWIN_PERLMUTTER_HOST`, `DTWIN_PERLMUTTER_BASE_DIR`,
`DTWIN_PERLMUTTER_BATCH_DIR`
- `DTWIN_GX_PATH`
- `DTWIN_FLUX_USER`, `DTWIN_FLUX_HOST`, `DTWIN_FLUX_BASE_DIR`, `DTWIN_FLUX_PYTHON`
- existing `SSH` controls: `DTWIN_SSH_IDENTITY`, `DTWIN_SSH_CONTROL_PATH`,
`DTWIN_SSH_CONTROL_PERSIST`, `DTWIN_SSH_CONNECT_TIMEOUT`

6 GUI Workflow

The Flask GUI still uses the route `/save_hpc_config` for compatibility, but the saved data now lands in the user-local config file instead of `db_analysis/hpc_config.json`.

The settings panel now captures:

- Perlmutter SSH and monitor settings,
- Perlmutter remote base and batch directories,
- GX installation path,
- Flux SSH user, host, base directory, and Python path.

This means one person can configure the same repository checkout for their own accounts without modifying repo-tracked files.

Source locations are intentionally excluded from this panel. Those belong to the database workflow metadata rather than to user-local runtime settings.

7 CLI and Shell Workflow

7.1 Python entrypoints

Active Python entrypoints import `dtwin_config.py` and resolve defaults from the user-local config before falling back to derived layout conventions.

Examples include:

- Perlmutter batch deployment and transfer scripts in `batch/`,
- ingestion scripts such as `db_update/populate_data_equil_from_Mate_KinEFIT.py`,
- the active Flux ingestion path under `db_update/Transp_full_auto/`.

7.2 Shell helper

The shell helper can export resolved settings:

```
python3 tools/resolve_dtwin_env.py --profile perlmutter --format shell
python3 tools/resolve_dtwin_env.py --profile flux --format shell
```

Typical usage in a shell script is:

```
eval "$(python3 "$DTWIN_ROOT/tools/resolve_dtwin_env.py" --profile flux --format shell)"
```

8 Flux Runtime Hand-Off

The Flux full-auto wrapper now works in two stages:

1. the laptop-side launcher resolves the Flux profile and writes `datamak_runtime.env` into the upload bundle,

2. the Flux-side launcher and helper scripts source that file before running Python entrypoints.

This file contains exported Flux settings such as:

- DTWIN_FLUX_USER
- DTWIN_FLUX_HOST
- DTWIN_FLUX_BASE_DIR
- DTWIN_FLUX_PYTHON

As a result, the active Flux scripts no longer need hardcoded paths like `/u/someuser/...`

9 Perlmutter Runtime Behavior

Perlmutter batch deployment now resolves remote host, base directory, batch directory, SSH controls, and GX path from the shared config layer.

The job submit scripts use:

```
GX_PATH="${DTWIN_GX_PATH:-$HOME/GX/gx_next6}"
```

When a custom GX path is configured, the deploy scripts pass it into `sbatch` through `-export=ALL,DTWIN_GX_PATH=...`

10 Migration from Legacy Repo-Local Config

Older GUI settings were written to:

```
db_analysis/hpc_config.json
```

The new config layer still reads that file as a legacy fallback if the user-local config does not yet exist. Once a user saves settings through the GUI, the new user-local config becomes the active source of truth.

This migration applies to Perlmutter and Flux runtime settings only. Source definitions should remain database-backed rather than being migrated into the GUI settings workflow.

11 Typical Setup for a New User

1. clone the repository,
2. create a Python environment and install the Datamak dependencies,
3. install or point to a working `pyrokinetics` environment,
4. open the GUI and save Perlmutter, Flux, and source-root settings,
5. verify SSH connectivity from the GUI using the `test/open` actions,
6. run the workflow scripts normally; they will resolve settings from the user-local config.

12 Troubleshooting

- If a script says a source root is missing, set the corresponding `DTWIN_*` variable or save it in the GUI settings panel.
- If Perlmutter commands fail non-interactively, confirm the SSH identity and control-path settings and ensure `sshproxy` is active if your site requires it.
- If Flux full-auto scripts cannot find Python, set `flux.python_bin` in the user config or export `DTWIN_FLUX_PYTHON`.
- If an old checkout still appears to use repo-local settings, remove or rename stale `db_analysis/hpc_config.json` after migrating to the user-local config.

13 Scope Note

This upgrade covers the active GUI, Perlmutter batch flow, active ingestion scripts, and `db_update/Transp_full_auto/`. Older or duplicate trees such as the top-level `transp_full_auto/` directory and `db_update/Transp_full_auto_remote/` remain legacy audit targets and should be cleaned up in a follow-up pass if they are reactivated.